



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

M.C. René Adrián Dávila Pérez

Asignatura:

Programación orientada a objetos

Grupo:

07

No de práctica(s):

04

Integrante(s):

424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada:

1

Semestre:

2027 - 1

Fecha de entrega:

25 de septiembre de 2026

Observaciones:

Calificación:

Índice

1. INTRODUCCIÓN	2
2. OBJETIVOS	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. MARCO TEÓRICO	2
3.1. El Paradigma Orientado a Objetos: Tipos de Datos Abstractos (TDA), Clases y Objetos	2
3.2. Instanciación, métodos constructores y referencia <code>this</code>	3
3.3. Diagrama de Clases UML	3
3.4. Colecciones en Java: la interfaz Map (HashMap vs TreeMap)	4
4. DESARROLLO	4
4.1. Ejercicio 1: Consulta sobre Destructores en Java con IA Generativa	4
4.2. Ejercicio 2: Aplicación Analizador de Palabras Duplicadas	5
5. RESULTADOS	7
5.1. Análisis de los Resultados Obtenidos	8
6. CONCLUSIONES	8
7. REFERENCIAS	9

1. INTRODUCCIÓN

En la materia de Programación Orientada a Objetos, el uso de clases y objetos es la base para estructurar cualquier programa en Java. Una clase nos sirve como plantilla para definir los datos y comportamientos que tendrá un objeto, permitiéndonos resolver problemas mediante la creación de componentes que interactúan entre sí dentro de nuestro código.

En este reporte documentamos el trabajo que realizamos para la Práctica 4. Primero, llevamos a cabo un ejercicio de consulta con herramientas de Inteligencia Artificial generativa para investigar la manera en que Java gestiona la destrucción de objetos y la memoria, analizando por qué no existen destructores explícitos como en otros lenguajes. En segundo lugar, desarrollamos una aplicación en Java que analiza oraciones para contar y mostrar el número de palabras duplicadas, utilizando la clase `AnalizadorPalabras` propuesta por el profesor, procesando las cadenas de texto con expresiones regulares y organizando la información mediante mapas (`HashMap` y `TreeMap`).

2. OBJETIVOS

2.1. Objetivo General

Aplicar los conceptos básicos de la programación orientada a objetos en un lenguaje de programación.

2.2. Objetivos Específicos

- Investigar, mediante el uso de Inteligencia Artificial generativa, cómo funciona el manejo de la memoria en Java y la razón por la que no se utilizan destructores explícitos.
- Construir la clase `AnalizadorPalabras` basándonos en el diagrama UML visto en clase, definiendo sus atributos públicos y sus métodos correspondientes.
- Aplicar el uso de expresiones regulares para limpiar la oración ingresada por el usuario, omitiendo signos de puntuación e ignorando las diferencias entre mayúsculas y minúsculas.
- Utilizar estructuras de datos tipo `HashMap` y `TreeMap` para registrar las frecuencias de las palabras y mostrar las repetidas tanto en el orden de lectura como en orden alfabético.
- Implementar la clase principal `Main` para recibir los datos ingresados desde la consola e invocar los métodos de nuestra clase analizadora.

3. MARCO TEÓRICO

Para fundamentar la resolución de los ejercicios de esta práctica, consultamos artículos técnicos en sitios de programación, los apuntes tomados durante la clase y la información del manual de prácticas de la página del laboratorio (lcp02 FI UNAM).

3.1. El Paradigma Orientado a Objetos: Tipos de Datos Abstractos (TDA), Clases y Objetos

La Programación Orientada a Objetos nos permite modelar problemas del mundo real mediante entidades llamadas objetos, los cuales integran datos y funcionalidades en un mismo bloque de código. Un Tipo de Dato Abstracto define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, dejando de lado los detalles de su implementación.

- **Objeto:** Es una encapsulación genérica de datos y de los procedimientos para manipularlos [1]. Todo objeto posee dos elementos clave:

- **Atributos:** Almacenan el estado o los datos propios del objeto en un momento dado durante la ejecución del programa [1].
- **Métodos (Comportamientos):** Son las funciones o procedimientos que definen las acciones que el objeto puede ejecutar o la forma en que modifica sus atributos [1].
- **Clase:** Es la plantilla, molde o categoría abstracta a partir de la cual se crean los objetos. Define la estructura de atributos y métodos que tendrán sus instancias [2], aunque cada objeto mantendrá sus propios valores de datos y una identidad independiente.

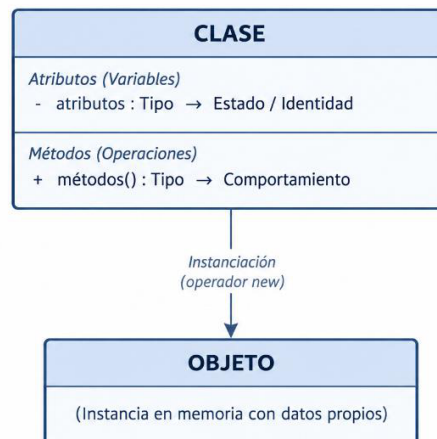


Figura 1: Representación de la relación entre una clase y un objeto mediante la instanciación con el operador `new`.

3.2. Instanciación, métodos constructores y referencia `this`

El proceso de crear un objeto a partir de una clase se conoce como instanciación. En Java, esto se logra mediante el operador `new`, el cual solicita espacio en la memoria Heap e invoca al método constructor de la clase [1]:

```
NombreDeLaClase objeto = new NombreDeLaClase();
```

La comunicación entre los componentes del programa sigue el modelo de paso de mensajes [1], donde un objeto ejecuta un método miembro mediante la sintaxis del punto:

```
objeto.nombreDelMétodo(parámetros);
```

- **Métodos Constructores:** Son bloques de código especiales encargados de inicializar los atributos del objeto al crearlo. Tienen exactamente el mismo nombre que la clase y no especifican ningún tipo de dato de retorno (no llevan la palabra `void`). Si no se escribe un constructor, Java asigna un constructor por defecto sin parámetros [1].
- **Palabra reservada `this`:** Es una variable de autorreferencia implícita que apunta al objeto actual que está ejecutando el código. Se utiliza principalmente para evitar ambigüedades cuando el nombre de un parámetro de un método es idéntico al nombre de un atributo de la clase [4].

3.3. Diagrama de Clases UML

El Lenguaje Unificado de Modelado (UML) nos permite representar visualmente la estructura de una clase mediante un diagrama rectangular dividido en tres niveles [1]:

1. **Nombre de la clase:** En la parte superior.
2. **Atributos:** Especificando visibilidad (+ público, - privado), identificador y tipo de dato.
3. **Métodos:** Mostrando visibilidad, nombre, lista de parámetros recibidos y tipo de dato devuelto.

3.4. Colecciones en Java: la interfaz Map (HashMap vs TreeMap)

En el framework de colecciones de Java (`java.util`), la interfaz `Map` nos permite almacenar información organizada en pares de clave y valor [5]. Una clave es un identificador único que no se puede repetir dentro del mapa, el cual nos da acceso a su valor correspondiente.

Para nuestro programa de conteo de palabras recurrimos a dos implementaciones de esta interfaz:

- **HashMap** [6]:
 - **Estructura Interna:** Funciona mediante una tabla Hash (arreglo de contenedores).
 - **Rendimiento:** Es extremadamente rápido; ofrece una complejidad temporal constante $O(1)$ para insertar `put` y buscar `get` elementos.
 - **Ordenamiento:** No guarda las palabras en un orden predecible; las organiza según el código Hash (`hashCode()`) generado para cada palabra.
- **TreeMap** [7]:
 - **Estructura Interna:** Está basado en un árbol auto-balanceable Rojo-Negro.
 - **Rendimiento:** Sus operaciones tienen una complejidad logarítmica $O(\log n)$.
 - **Ordenamiento:** Mantiene todas sus claves ordenadas de forma natural (orden alfabético/lexicográfico para cadenas de texto `String`). En nuestro código lo usamos para convertir los datos del `HashMap` y mostrar las palabras duplicadas ordenadas alfabéticamente de forma muy sencilla.

4. DESARROLLO

4.1. Ejercicio 1: Consulta sobre destructores en Java con IA Generativa

Cada uno de los 5 integrantes de nuestro equipo consultó una herramienta de IA generativa distinta utilizando el mismo prompt técnico para comparar las respuestas.

Prompt formulado por el equipo:

“Explica de manera clara y concisa: 1) ¿Qué es un destructor en Java o cuál es su equivalente?, 2) ¿Cómo funciona la liberación de memoria en Java?, 3) Muestra un ejemplo sencillo de cómo se maneja esto en código.”

Resumen de la información obtenida:

Comprobamos que en Java no existen los destructores explícitos como los que hay en lenguajes como C++.

- **Liberación de memoria:** Java administra la memoria de forma automática mediante el Recolector de Basura, el cual se encarga de eliminar de la memoria aquellos objetos que ya no están siendo utilizados por el programa.
- **Uso de `finalize()` y alternativas:** Anteriormente existía el método `finalize()`, pero las herramientas nos indicaron que ya no se utiliza porque era poco confiable. Actualmente,

si se necesita liberar recursos (como cerrar un archivo o una conexión), lo recomendable es implementar la interfaz `AutoCloseable` y usar la estructura `try-with-resources`.

4.2. Ejercicio 2: Aplicación Analizador de Palabras Duplicadas

Tomando como guía el diagrama UML que nos proporcionó el profesor, estructuramos la solución en dos archivos Java: `AnalizadorPalabras.java` y `Main.java`.

4.2.1. Lógica de la clase `AnalizadorPalabras`

La clase cuenta con dos atributos públicos: `oracion` (de tipo `String`) y `frecuencias` (de tipo `Map`). Sus métodos funcionan de la siguiente manera:

1. **Constructor `AnalizadorPalabras(String nuevaOracion)`:** Recibe la frase ingresada por el usuario, la guarda en la variable `oracion` e inicializa el mapa `frecuencias` como un `HashMap` vacío.
2. **Método `contarPalabras()`:**
 - Primero convierte toda la oración a minúsculas con `.toLowerCase()` para evitar diferencias entre letras mayúsculas y minúsculas.
 - Después limpia la frase usando la expresión regular siguiente, la cual quita comas, puntos y signos de puntuación dejando únicamente letras y números:

```
replaceAll("[^a-zA-Z0-9]", " ")
```
 - Separa la frase en palabras individuales usando:

```
split("\\s+")
```
 - Recorre la lista de palabras y las agrega al mapa `frecuencias`. Si la palabra ya existe en el mapa le suma 1 a su valor, y si es nueva la registra con un 1 usando el método `getOrDefault()`.
3. **Método `obtenerNumeroDuplicados()`:** Recorre únicamente los valores guardados en el mapa y cuenta cuántas palabras tienen una frecuencia mayor a 1, regresando dicho total.
4. **Método `mostrarDuplicadas(boolean ordenar)`:**
 - Si el parámetro `ordenar` es `true`, crea un `TreeMap` a partir de nuestro `HashMap` para que las palabras se acomoden en orden alfabético.
 - Imprime las palabras cuya frecuencia sea mayor a 1.
 - En caso de que ninguna palabra se haya repetido, muestra un mensaje indicando que no hubo duplicados.

4.2.2. Lógica de la clase `Main`

En la clase principal únicamente coordinamos la interacción con el usuario:

1. Leemos la oración introducida por teclado usando `Scanner`.
2. Instanciamos la clase `AnalizadorPalabras` pasándole la oración recibida.
3. Llamamos al método `contarPalabras()` para procesar el texto.
4. Obtenemos e imprimimos el total de palabras duplicadas.
5. Mostramos el detalle de las palabras repetidas primero sin ordenar y posteriormente ordenadas alfabéticamente pasando el parámetro correspondiente a `mostrarDuplicadas()`.

4.2.3. Diagrama de clases UML

La Figura 2 muestra la relación de uso entre las clases `Main` y `AnalizadorPalabras`, así como sus atributos y métodos.

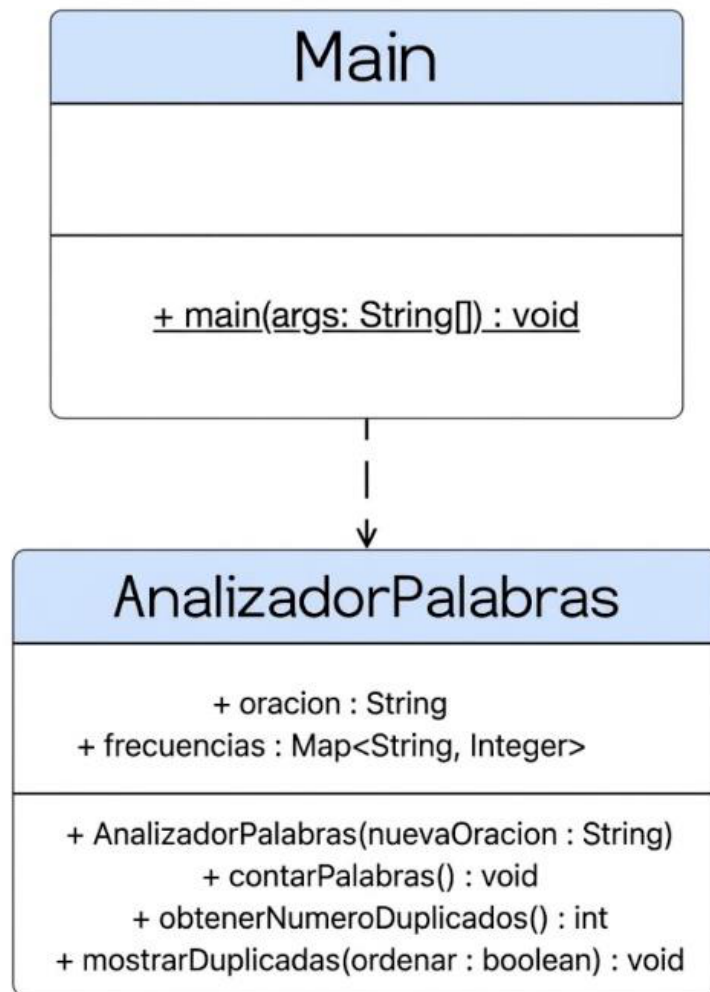


Figura 2: Diagrama UML de las clases `Main` y `AnalizadorPalabras`, detallando sus atributos, métodos y la relación de uso entre ambas.

4.2.4. Casos de Prueba

Antes de hacer la entrega probamos el funcionamiento del programa con los siguientes casos de prueba:

Caso 1: Oración con signos de puntuación y mayúsculas

- **Entrada:** “El perro corre en el parque, el perro salta alto. ¡El perro es muy rápido!”
- **Salida esperada:**
 - Palabras diferentes que se duplicaron: 2.
 - Detalle sin ordenar: `e1`: 3 veces, `perro`: 3 veces.
 - Detalle ordenado alfabéticamente: `e1`: 3 veces, `perro`: 3 veces.

Caso 2: Oración sin palabras duplicadas

- **Entrada:** “Hola a todos, somos la brigada 1.”

■ **Salida esperada:**

- Palabras diferentes que se duplicaron: 0.
- Mensaje: “No se encontraron palabras duplicadas en la oracion.”

5. RESULTADOS

A continuación, presentamos la evidencia del correcto funcionamiento de nuestro programa en la consola de comandos de Linux, validando el comportamiento de las clases `AnalizadorPalabras` y `Main`.

```
POO-E1-T-/P4 on | main [?] via ☕ v21.0.12
> javac Main.java

POO-E1-T-/P4 on | main [?] via ☕ v21.0.12 took 2s
> java Main
Ingresa la oracion a analizar
hola HOLA HoLa ahoL ola ola alo

Palabras diferentes que se duplicaron: 2

Detalle de duplicados(Sin ordenar)
- ola: 2 veces
- hola: 3 veces

Detalle de duplicados (Orden alfabetico)
- hola: 3 veces
- ola: 2 veces
```

Figura 3: Captura de pantalla de la terminal mostrando la compilación con `javac` y la ejecución del programa analizando una cadena con distintas variaciones de mayúsculas/minúsculas y repeticiones de palabras.

```
> java Main
Ingresa la oracion a analizar
Esto es un ejemplo sin repeticiones ok ?

Palabras diferentes que se duplicaron: 0

Detalle de duplicados(Sin ordenar)
No se encontraron palabras duplicadas en la oracion.

Detalle de duplicados (Orden alfabetico)
No se encontraron palabras duplicadas en la oracion.
```

Figura 4: Captura de pantalla de la terminal mostrando el comportamiento del programa al recibir una oración donde no existen palabras repetidas y cuenta con un signo de puntuación aislante (?).

5.1. Análisis de los Resultados Obtenidos

Al evaluar los resultados mostrados en las Figuras 3 y 4, confirmamos la correcta implementación de la lógica algorítmica y la manipulación de colecciones dentro de Java:

1. **Insensibilidad a Mayúsculas/Minúsculas:** En la Figura 3, la cadena ingresada incluye las palabras “hola”, “HOLA” y “HoLa”. El método `contarPalabras()` convirtió correctamente toda la entrada a minúsculas mediante `toLowerCase()`, contabilizando las 3 variaciones como una sola clave “hola” con frecuencia 3. Las palabras “ahol” y “alo” no se contaron como duplicadas al ser cadenas ortográficamente distintas.
2. **Depuración de Signos de Puntuación:** En la Figura 4, la oración ingresada termina con un signo de interrogación (“?”). La expresión regular mostrada en el desarrollo removió de forma precisa este carácter especial, evitando que afectara la palabra “ok” o que fuera interpretado erróneamente como un token independiente en la separación por espacios.
3. **Manejo de Respuestas Nulas:** Cuando no existen palabras repetidas (como se observa en la Figura 4), el método `obtenerNumeroDuplicados()` retorna 0 y la bandera dentro de `mostrarDuplicadas()` activa de manera adecuada el mensaje informativo “No se encontraron palabras duplicadas en la oracion.” en ambas impresiones.
4. **Comportamiento de HashMap vs TreeMap:** En la Figura 3, observamos claramente la diferencia estructural de los mapas:
 - **Sin ordenar (HashMap):** Muestra los duplicados en el orden determinado por la tabla de dispersión de Java (ola primero y hola después).
 - **Orden alfabético (TreeMap):** Al instanciar dinámicamente un `TreeMap` con el contenido de las frecuencias, los elementos se reordenaron de forma automática en estricto orden lexicográfico (hola antes que ola).

6. CONCLUSIONES

En esta práctica reforzamos la forma de trabajar con clases y objetos en Java, entendiendo cómo separar la lógica de un programa dentro de una clase específica para luego utilizarla desde la clase principal. A través de la actividad de investigación con la IA, nos quedó claro que Java no requiere del uso de destructores manuales debido a que la liberación de memoria se realiza de forma automática.

Asimismo, la implementación del analizador de palabras nos permitió poner en práctica el uso de mapas para resolver problemas de conteo de frecuencias, así como la manipulación de cadenas de texto mediante expresiones regulares para limpiar la entrada del usuario. Logramos cumplir con todos los objetivos planteados al inicio de la sesión y el programa respondió de acuerdo con lo solicitado en el diagrama UML.

Referencias

- [1] Laboratorio de Computación de la Facultad de Ingeniería (lcp02 FI UNAM), “Manual de Prácticas de Programación Orientada a Objetos - Práctica 4: Clases y Objetos”, Universidad Nacional Autónoma de México, 2026.
- [2] Coordinación de Universidad Abierta, Innovación Educativa y Educación a Distancia (CUAED UNAM), “Clases y Objetos en Programación Orientada a Objetos”, Unidad de Apoyo para el Aprendizaje (UAPA), UNAM. [En línea]. Disponible en: [Enlace UAPA](#)
- [3] OpenWebinars, “Introducción a POO en Java: Objetos y clases”, Blog de OpenWebinars. [En línea]. Disponible en: [Enlace OpenWebinars](#)
- [4] CiberAula, “Uso de la palabra reservada this en Java”, Cursos de Programación Java. [En línea]. Disponible en: [Enlace CiberAula](#)
- [5] CodeGym, “La interfaz Map en Java y sus aplicaciones”, Artículos y Comunidad CodeGym. [En línea]. Disponible en: [Enlace CodeGym](#)
- [6] W3Schools, “Java HashMap”, W3Schools Java Tutorial. [En línea]. Disponible en: [Enlace W3Schools](#)
- [7] W3Schools, “Java TreeMap”, W3Schools Java Tutorial. [En línea]. Disponible en: [Enlace W3Schools](#)